AI-Powered Sanskrit NLP Framework for Morphological and Semantic Analysis

IT Project Management (21MGH303P)

B.Tech – III Year | 6th Semester

(AI-driven framework for morphological analysis, semantic parsing, and structured knowledge extraction from classical Sanskrit texts based on Paninian grammar.)

Prepared By

Aditya Kumar Kapar (RA2311042030002)

Milan Kuiry (RA2311042030008)

Harsh Garg (RA2311042030010)

Institution

SRM Institute of Science and Technology

Delhi-NCR Campus, Modinagar, Uttar Pradesh

Problem Statement

- **Current Challenges in Sanskrit NLP**
  - Lack of integrated systems combining Paninian grammar with modern AI techniques
  - Existing tools are primarily:
    - Rule-based (limited scalability)
    - Or statistical (lack linguistic depth)
- **Technical Limitations**
  - Inefficient morphological analysis for complex Sanskrit structures
  - Absence of robust semantic parsing frameworks
  - Limited capability for structured knowledge extraction
- **Research & Practical Gaps**
  - Minimal support for large-scale corpus processing
  - Poor accessibility for researchers (lack of standardized APIs)
  - High dependency on manual linguistic analysis

Project Objectives

Primary Objectives

- Develop a scalable NLP framework for Sanskrit text analysis
- Ensure integration of Paninian grammar with AI techniques

Management Objectives

- Deliver project within:
- Budget: ₹33–41 Lakhs
- Timeline: 218 Days
- Maintain quality standards (≥92% accuracy target)

Operational Objectives

- Provide API-based access for researchers
- Enable large-scale corpus processing
- Ensure ease of use and adoption

Strategic Goal

- Establish a sustainable and extensible platform for academic and research use

Requirements & Scope Overview

Requirement Approach

- **Requirements gathered via:**
  - Stakeholder workshops
  - Document analysis
  - Institutional survey

Key Requirement Categories

- Functional: Core system capabilities (analysis, API access, processing)
- Non-Functional: Performance, accuracy, scalability, reliability

Scope Definition

In Scope

- NLP framework development
- API deployment and documentation
- Cloud-based processing

Out of Scope

- Mobile application
- Real-time streaming systems
- OCR / speech processing

Priority Management

- Requirements prioritized using MoSCoW method
- Focus on critical deliverables for initial release

Stakeholder Management

Key Stakeholders

- Sponsors (UGC / MHRD)
- Sanskrit Linguists
- NLP Engineers & Data Team
- Researchers & End Users
- Cloud Service Provider

Communication Strategy

- Weekly reviews (core team)
- Monthly reporting (sponsors)
- Feedback cycles (end users)

Management Approach

- Stakeholders classified based on Power vs Interest
- Engagement strategies:
  - High Power–High Interest → Close monitoring
  - Others → periodic communication

Outcome

- Improved decision-making
- Reduced project risks
- Better stakeholder alignment

Stakeholder Classification Framework

Stakeholders are mapped based on:

- Power (influence on project)
- Interest (level of involvement)

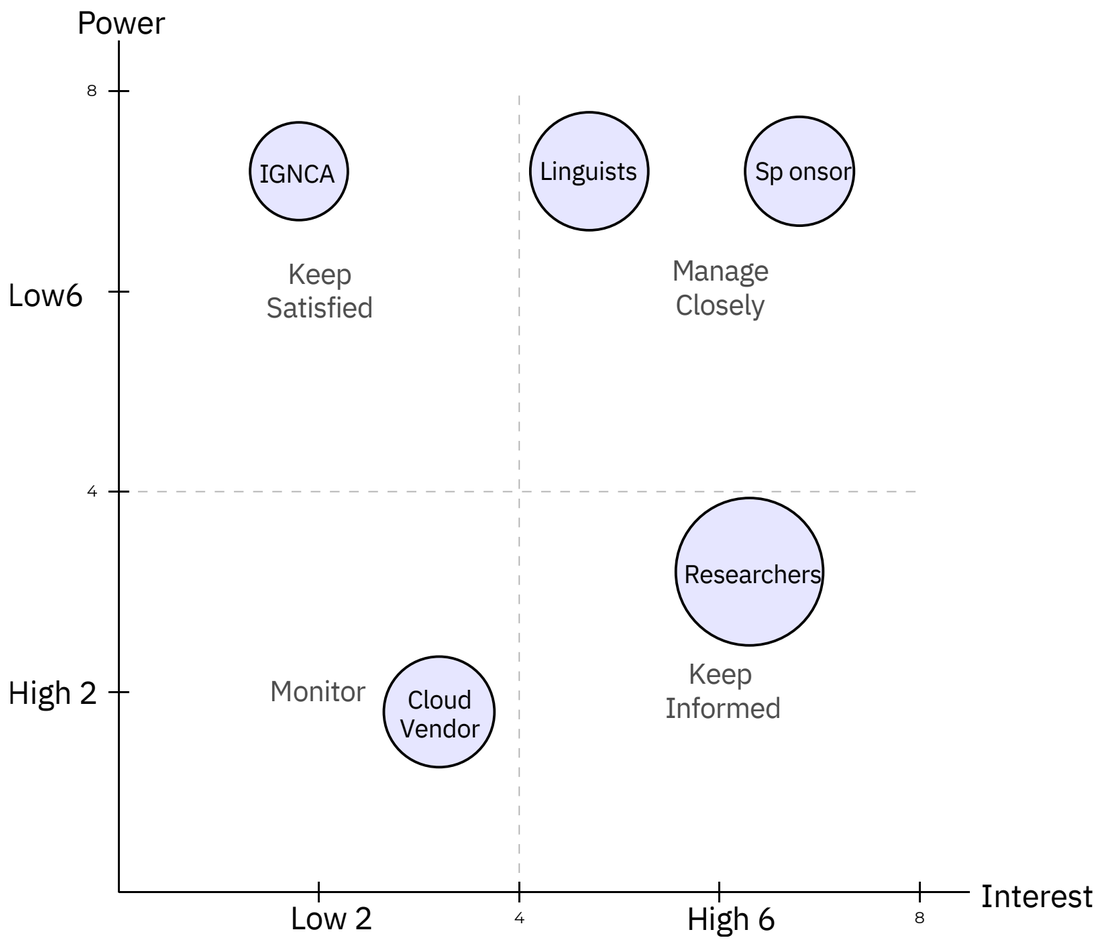
Management Strategy

- **High Power – High Interest:** Manage closely
- **High Power – Low Interest:** Keep satisfied
- **Low Power – High Interest:** Keep informed
- **Low Power – Low Interest:** Monitor

Project Insight

- **Critical stakeholders (Sponsors, Linguists) require continuous engagement**
- **End users require regular updates and feedback integration**

Stakeholder Power/Interest Grid



Requirement Prioritization (MoSCoW)

Purpose

- Ensure focused **delivery within time** and budget constraints
- Prioritize requirements based on **business value and criticality**

Categories

- **Must Have:** Essential for project success
- **Should Have:** Important but not critical
- **Could Have:** Value-adding enhancements
- **Won't Have:** Out of current scope

Management Insight

- Enables **controlled scope and prevents scope creep**
- Supports **phased delivery** strategy

Requirement Prioritization (MoSCoW)

Must Have - Morphological Analyzer - Semantic Parser - Core NLP Engine	Should Have - REST API - Batch Processing - Knowledge Extraction
Could Have - UI Visualization - Bilingual Interface	Won’t Have -OCR - Speech Processing - Translation

Work Breakdown Structure (WBS)

Purpose

- Decompose the project into **manageable** components
- Improve **planning, scheduling, and resource allocation**

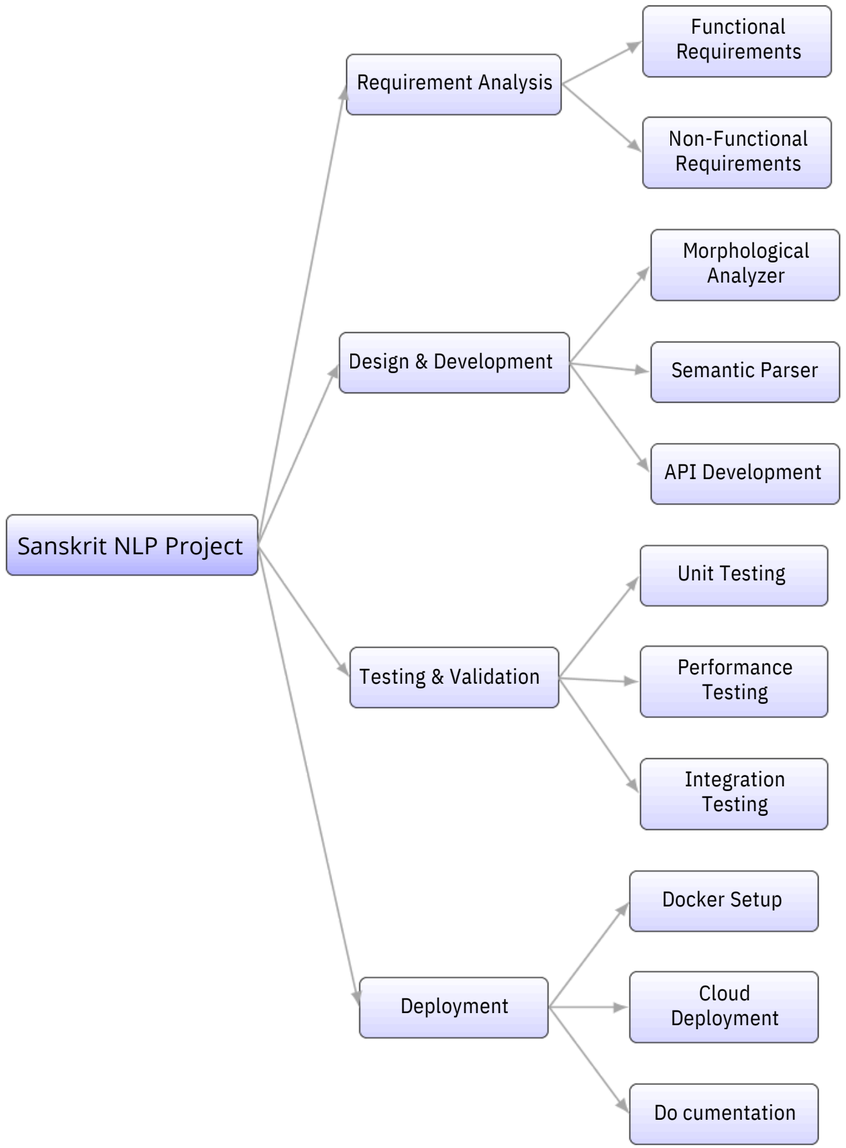
Structure

- **Level 1:** Project
- **Level 2:** Major phases **(Analysis, Development, Testing, Deployment)**
- **Level 3:** Subtasks

Management Insight

- Enables **clear task ownership**
- Supports accurate **time and cost estimation**
- Forms the basis for **PERT, CPM, and scheduling**

Work Breakdown Structure (WBS)



Critical Path Method (CPM) Analysis

Purpose

- Identify the **longest path of dependent activities**
- Determine **minimum project completion time**

Key Insight

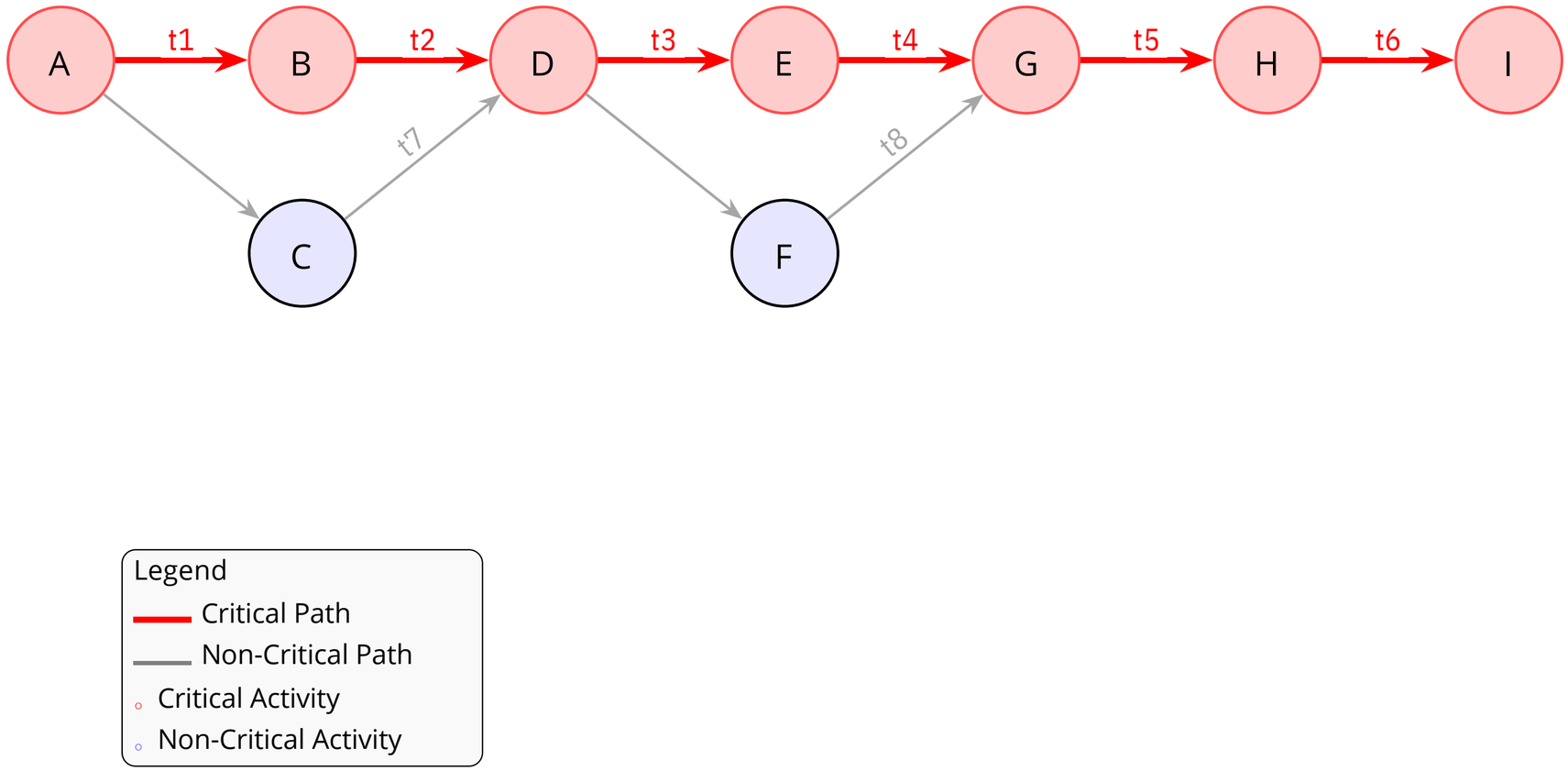
- **Critical Path: A → B → D → E → G → H → I**
- **Total Duration: 218 Days**

Management Implications

- Activities on critical path have zero slack
- Any delay directly impacts project timeline
- Enables focused monitoring and control

Critical Path Method (CPM) Analysis

Critical Path Method – Project Network



Project Schedule

Purpose

- Visualize **project timeline and task sequencing**
- Track **start and end dates of activities**

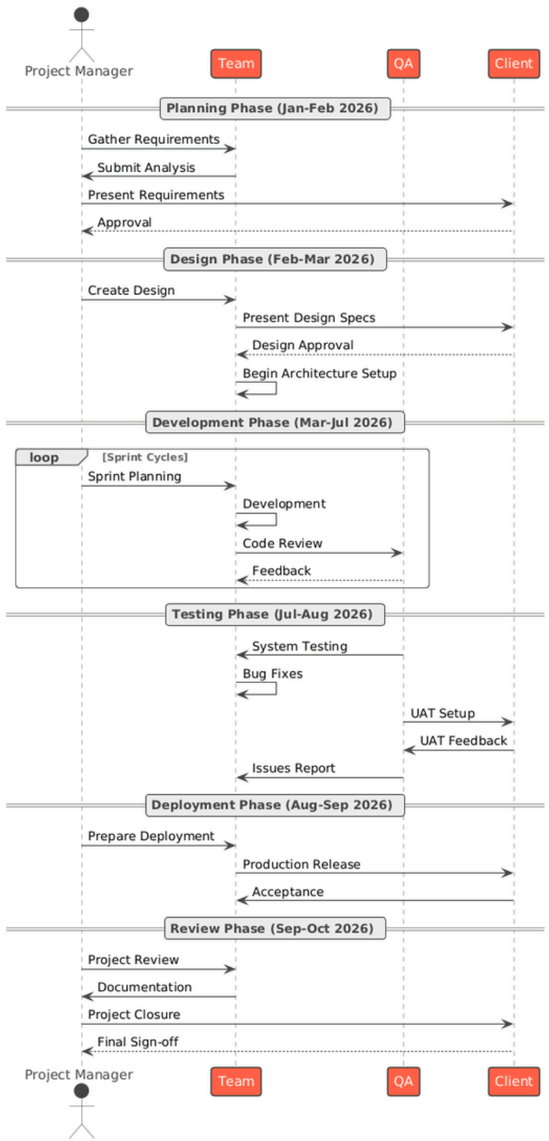
Schedule Overview

- **Total Duration: 218 Days (~10.5 Months)**
- **Phases:**
  - **Requirement Analysis**
  - **Design & Development**
  - **Testing**
  - **Deployment**

Management Insight

- Enables progress tracking against plan
- Helps identify overlapping tasks and delays
- Supports resource planning and coordination

Project Schedule(Gantt Chart)



Project Performance Monitoring

Objective

- **Track project performance across:**
  - **Cost**
  - **Schedule**
  - **Scope**

Monitoring Approach

- Use of **Key Performance Indicators (KPIs)**
- Regular **status reviews and reporting**
- Comparison of **planned vs actual performance**

Control Mechanisms

- **Variance analysis**
- **Corrective action planning**
- **Continuous performance evaluation**

Management Insight

- Early identification of **delays and cost overruns**
- Enables **data-driven** decision making
- Ensures project remains aligned with **baseline plan**

Earned Value Analysis (EVA)

Purpose

- Evaluate project **performance** using **cost and schedule metrics**
- Compare **planned vs actual vs earned** progress

Key Metrics

- **Planned Value (PV):** Budgeted work
- **Earned Value (EV):** Work completed
- **Actual Cost (AC):** Cost incurred

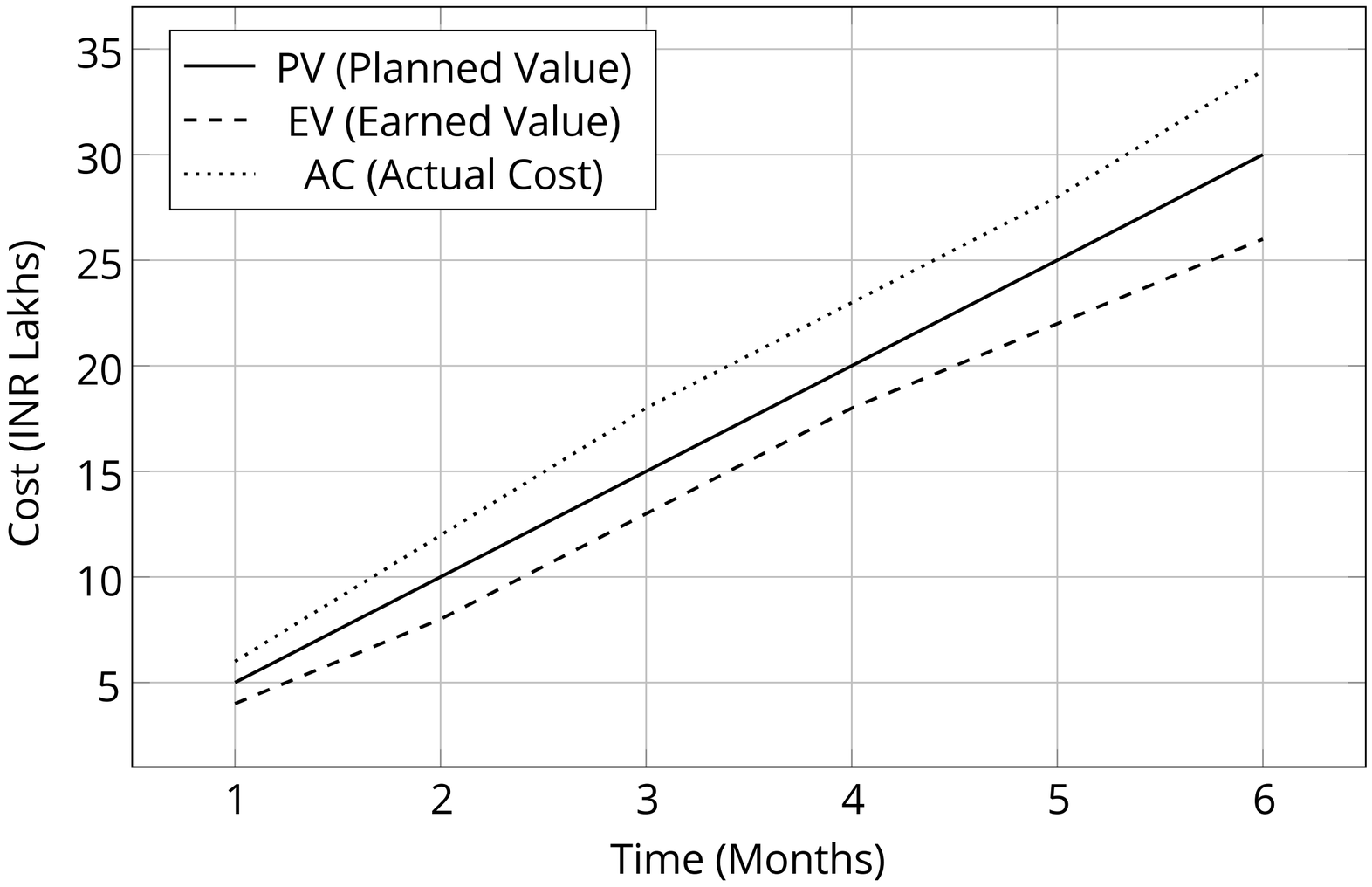
Performance Indicators

- **CPI (Cost Performance Index)**
- **SPI (Schedule Performance Index)**

Management Insight

- **EV < PV → Schedule delay**
- **EV < AC → Cost overrun**
- Supports forecasting and corrective actions

Earned Value Analysis (EVA)



Project Risk Management

Objective

- **Identify, assess, and control** project risks proactively

Risk Management Process

- Risk Identification
- Risk Analysis (Probability & Impact)
- Risk Response Planning
- Risk Monitoring

Key Risks Identified

- Complexity of linguistic rules
- Resource constraints (expert availability)
- Data annotation delays

Management Strategies

- **Risk mitigation** and **contingency** planning
- Allocation of risk reserves
- Continuous monitoring through **RMMM plan**

Outcome

- Reduced uncertainty
- Improved project stability and predictability

Risk Assessment Matrix

Project Risks

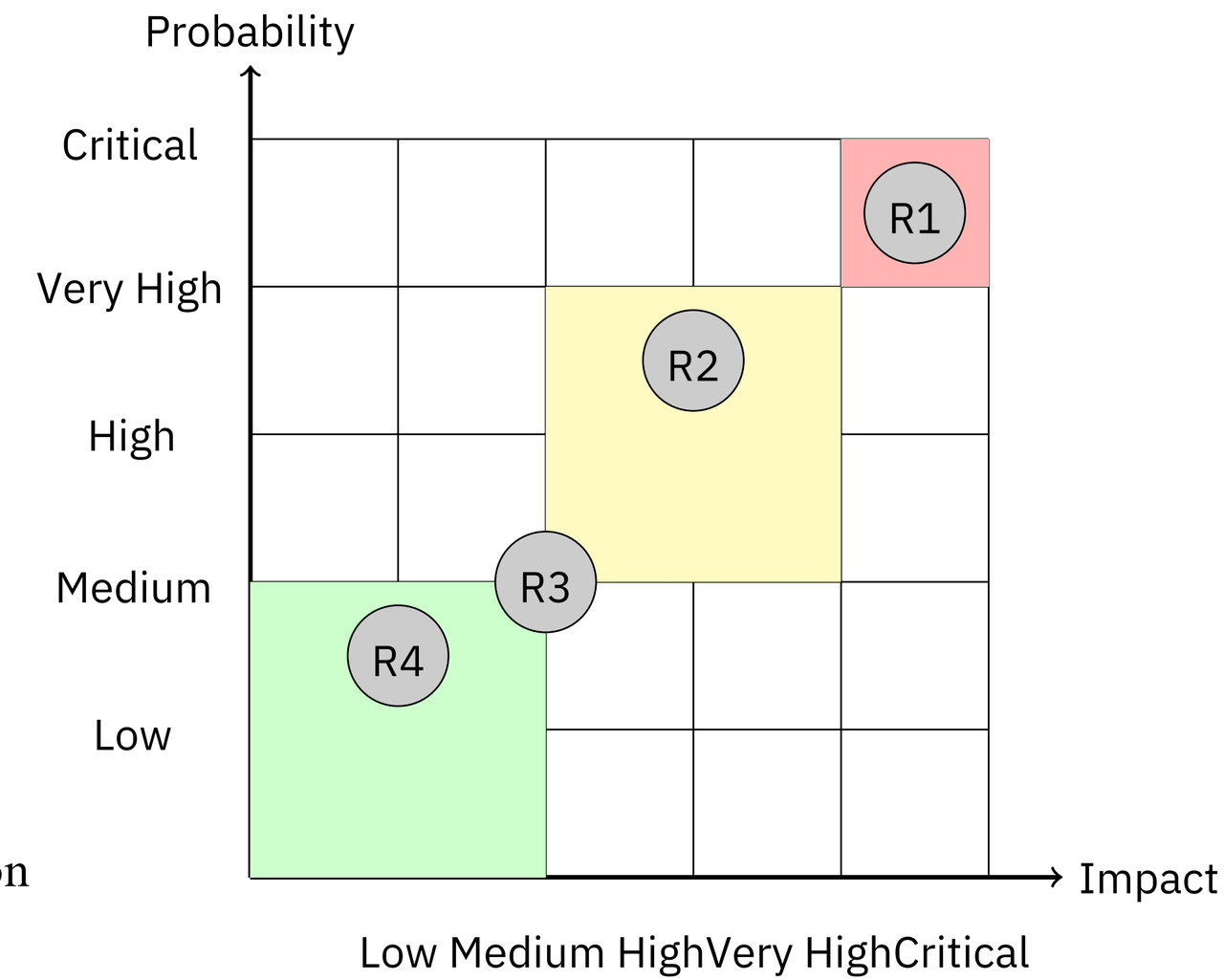
- **R1:** Paninian grammar complexity
- **R2:** Limited expert availability
- **R3:** Data annotation delays
- **R4:** Minor operational inefficiencies

Risk Positioning

- **R1 → Critical Zone**
  - High probability + very high impact
- **R2 → High Risk Zone**
  - Moderate–high probability and impact
- **R3 → Medium Risk Zone**
  - Moderate impact, manageable
- **R4 → Low Risk Zone**
  - Low probability and impact

Management Insight

- Focus on **R1** and **R2** for immediate mitigation
- Monitor **R3**, accept/manage **R4**



Agile Approach

Rationale for Agile Adoption

- **Evolving requirements** due to **complex** Sanskrit grammar rules
- Need for **iterative validation** with linguists
- **Continuous feedback** from research stakeholders

Framework Used

- Scrum **(2-week sprints)** for **development & testing**
- Integrated with **XP** practices for quality

Sprint Structure (Project Context)

- **Sprint Duration: 2 Weeks**
- **Activities:**
  - Requirement refinement (with linguists)
  - Feature development (NLP modules)
  - Testing & validation
  - Review with stakeholders

Scrum Framework

Workflow in This Project

- **Product Backlog: NLP features (morphology, parsing, API)**
- **Sprint Planning:** Prioritization based on **MoSCoW**
- **Sprint Backlog:** Selected tasks for each sprint
- **Sprint (2 Weeks):** Development + testing cycle

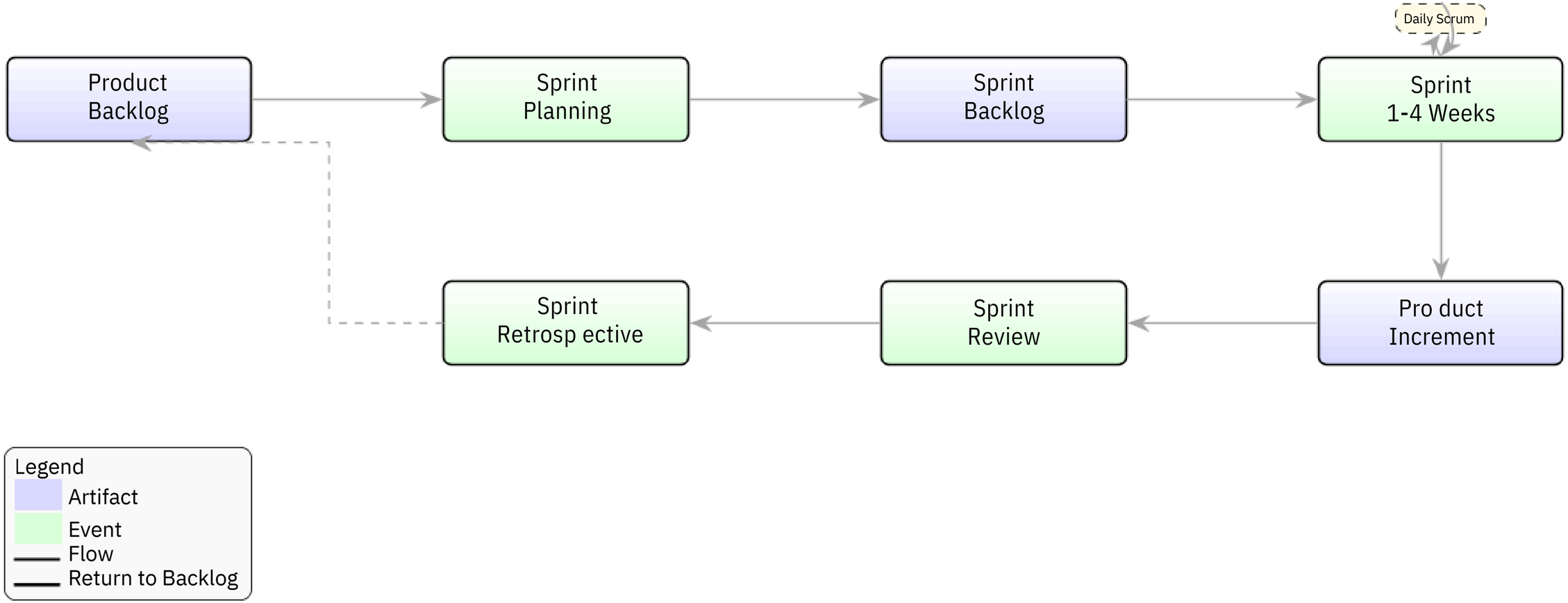
Execution Cycle

- **Daily Scrum → progress tracking**
- **Sprint Review → stakeholder validation (linguists)**
- **Sprint Retrospective → process improvement**
- **Output → Incremental feature delivery**

Project-Specific Insight

- Ensures continuous validation of linguistic accuracy
- Supports iterative delivery of NLP modules
- Improves stakeholder collaboration

Scrum Framework



Sprint Progress Tracking

Sprint Context

- **Sprint Duration: 10 Days (2 Weeks)**
- **Initial Workload: 50 Story Points**
- **Scope: NLP module development + testing**

Performance Observation

- Actual progress remains above ideal line
- Indicates slower completion rate

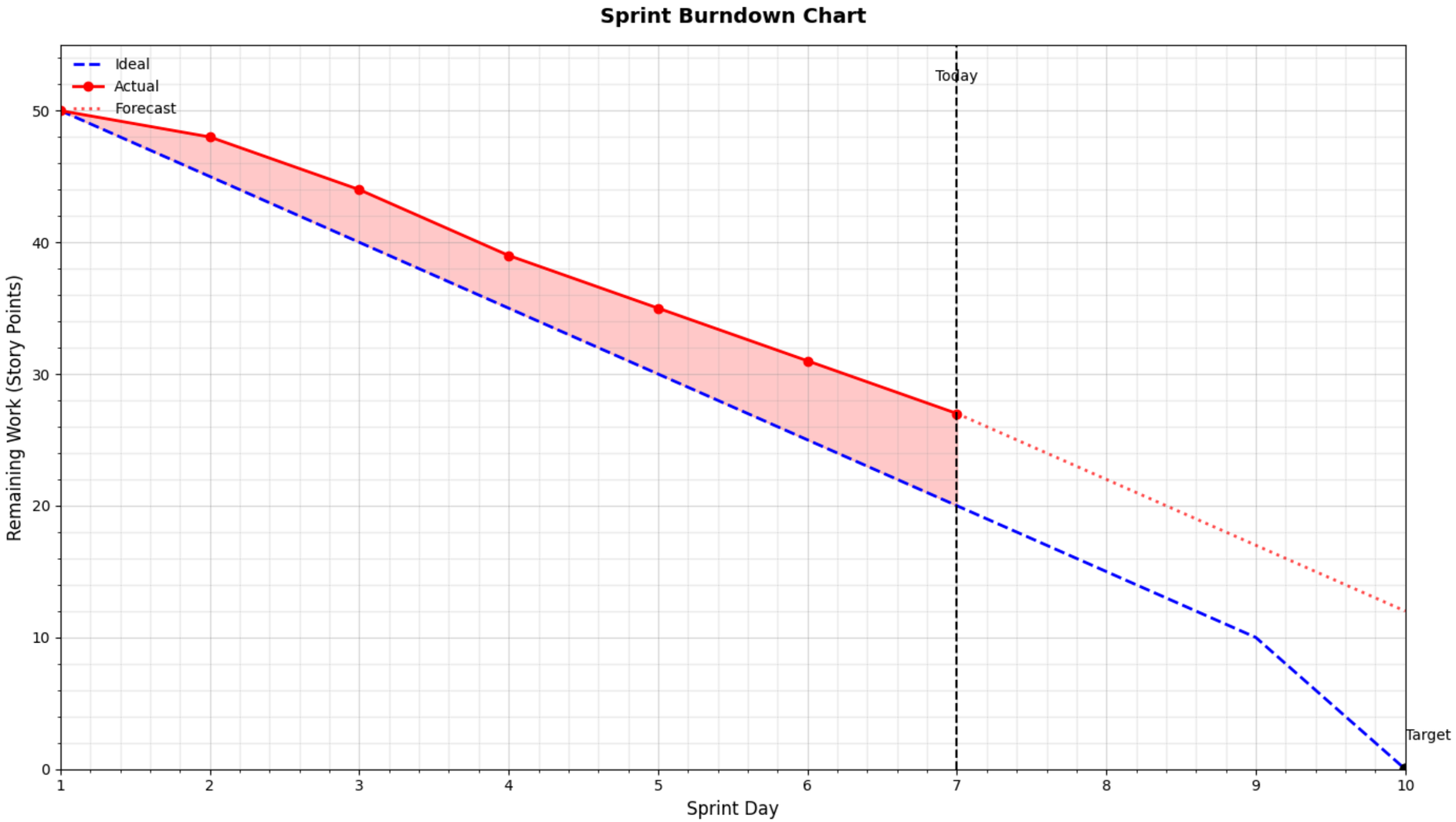
Forecast Insight

- Remaining work at sprint end → ~12 story points
- Risk of spillover to next sprint

Management Action

- Reallocate resources for critical tasks
- Improve task breakdown in next sprint
- Increase monitoring through daily scrums

Sprint Progress Tracking



DevOps & CI/CD Implementation

Objective

- Ensure reliable deployment of NLP framework
- Enable continuous integration of linguistic updates

Pipeline in This Project

- **Code Commit:** NLP modules (morphology, parser, API) via **GitHub**
- **Build: Integration of rule-based + ML components**
- **Test:**
  - **Morphological accuracy validation**
  - **API response testing (< 2 sec target)**
- **Package: Docker container** with NLP environment
- **Deploy: Cloud deployment** for research access

Project Insight

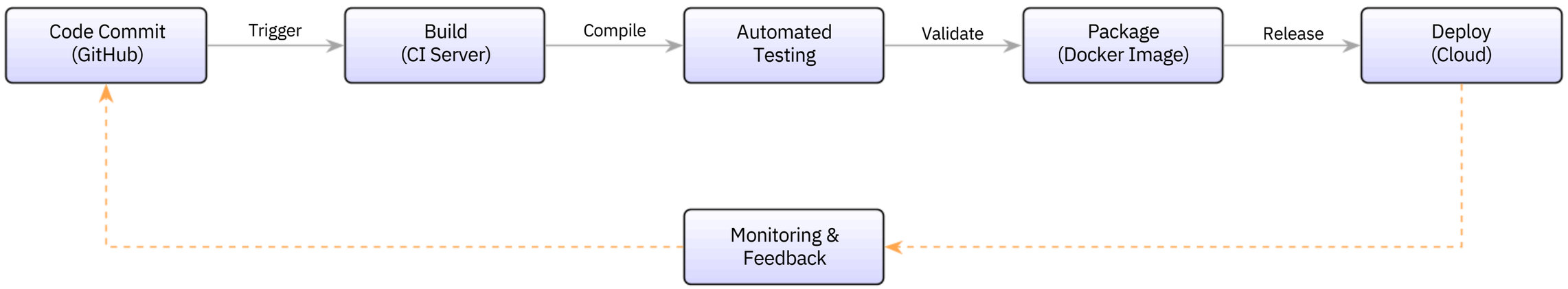
- Supports frequent updates of grammar rules (Panini sutras)
- Ensures consistent execution across environments
- Enables rapid bug fixes and feature releases

Management Impact

- **Reduces** deployment risk
- Improves delivery speed
- Enhances system reliability for stakeholders

DevOps & CI/CD Implementation

CI/CD Pipeline



Project Results & Outcomes

Project Performance

- **Total Cost: ₹33–41 Lakhs**
- **Project Duration: 218 Days (~10.5 Months)**
- **Payback Period: 3–4 Years**

Financial Evaluation

- **Net Present Value (NPV): Positive (~₹12.6 Lakhs)**
- **Indicates financial feasibility and long-term viability**

Operational Outcomes

- Achieved target accuracy (**~92%**) for analysis
- **Successful development of:**
  - **Morphological analyzer**
  - **Semantic parser**
  - **API-based system**

Management Outcome

- Delivered within planned scope, cost, and schedule
- **Effective use of:**
  - **Agile (Scrum)**
  - **Risk management**
  - **CI/CD practices**

Overall Impact

- Scalable platform for academic and research use
- Supports digital preservation and linguistic analysis

Conclusion

Project Summary

- Successfully planned and managed an AI-driven Sanskrit NLP framework
- Integrated traditional linguistic rules with modern technologies

Key Achievements

- Effective control of time, cost, and scope
- **Implementation of:**
  - **Structured planning (WBS, CPM)**
  - **Monitoring (EVA, Risk Management)**
  - **Agile & DevOps practices**

Managerial Insight

- Importance of **stakeholder collaboration (linguists + engineers)**
- Need for **adaptive planning in complex domains**
- Value of **continuous monitoring and iterative delivery**

Thank You!